

EXPERIMENT – 7

AIM:

To build an Artificial Neural Network (ANN) using Keras/TensorFlow for classification on a real-world dataset.

1. Dataset Source

- **Dataset:** Churn Modelling Dataset
- **Source:** Kaggle
- **Link:** <https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling>
- **File Used:** Churn_Modelling.csv

2. Dataset Description

The dataset contains customer information from a bank and is used to predict whether a customer will leave the bank (churn) or not.

Target Variable:

- **Exited**
 - 1 → Customer left the bank
 - 0 → Customer stayed

Predictor Variables Used:

- CreditScore
- Geography
- Gender
- Age
- Tenure
- Balance
- NumOfProducts
- HasCrCard
- IsActiveMember
- EstimatedSalary

Data Preprocessing:

- Removed unnecessary columns like RowNumber, CustomerId, Surname
- Converted categorical variables (Gender, Geography) into numerical form
- Applied one-hot encoding
- Performed feature scaling using StandardScaler

3. Mathematical Formulation of Artificial Neural Network

An Artificial Neural Network consists of interconnected layers of neurons.

Each neuron performs:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Output is passed through an activation function.

Activation Functions:

- **ReLU (Hidden Layers):**

$$f(x) = \max(0, x)$$

- **Sigmoid (Output Layer):**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Loss Function:

Binary Cross Entropy:

$$Loss = -[y \log(p) + (1 - y) \log(1 - p)]$$

Optimization Algorithm:

- Adam Optimizer is used for efficient weight updates.

Network Architecture:

- Input Layer: Number of features
- Hidden Layer 1: 64 neurons (ReLU)

- Dropout Layer (0.3)
- Hidden Layer 2: 32 neurons (ReLU)
- Dropout Layer (0.3)
- Hidden Layer 3: 16 neurons (ReLU)
- Output Layer: 1 neuron (Sigmoid)

4. Algorithm Limitations

- Requires large dataset for better performance
- Training time is higher
- Sensitive to hyperparameters
- Less interpretable compared to traditional models
- Performance affected by class imbalance

5. Methodology / Workflow

- Import required libraries
- Load dataset
- Perform data preprocessing
- Encode categorical variables
- Split dataset into training and testing sets
- Apply feature scaling
- Build ANN model using Keras
- Train the model using training data
- Monitor training and validation performance
- Predict test data results
- Evaluate model using accuracy, confusion matrix, classification report, and ROC-AUC
- Visualize results using graphs

6. Performance Analysis

- The ANN model achieved good accuracy on the test dataset
- Training and validation graphs show learning behavior of the model
- Confusion matrix indicates correct classification of majority class
- ROC-AUC score indicates model's classification ability

Observation:

- Model performs well on majority class
- Some misclassification occurs due to data imbalance

7. Hyperparameter Tuning

Parameters used:

- Optimizer: Adam
- Loss Function: Binary Cross Entropy
- Epochs: 30
- Batch Size: 32
- Activation Functions: ReLU and Sigmoid
- Dropout: 0.3

Tuning improves generalization and reduces overfitting.

CODE:

1. Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

2. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
roc_auc_score, roc_curve
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

3. Load Dataset

```
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
```

```
print("Dataset Shape:", df.shape)
print(df.head())
```

4. Data Preprocessing (for Churn dataset)

```

X = df.iloc[:, 3:-1]
y = df.iloc[:, -1]

# Convert categorical to numeric
X = pd.get_dummies(X, drop_first=True)

# Train Test Split (Stratified like your PDF)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature Scaling
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 5. Build ANN Model (Same as PDF architecture)
model = Sequential()

model.add(Dense(64, activation='relu', input_shape=(X_train_s.shape[1],)))
model.add(Dropout(0.3))

model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(16, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

# Compile Model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

# 6. Train Model
history = model.fit(

```

```
X_train_s, y_train,  
validation_split=0.2,  
epochs=30,  
batch_size=32,  
verbose=1  
)
```

7. Plot Training Graphs (LIKE PAGE 9)

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title("Training vs Validation Loss")  
plt.legend()
```

```
plt.subplot(1,2,2)  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title("Training vs Validation Accuracy")  
plt.legend()
```

```
plt.show()
```

8. Predictions

```
y_prob = model.predict(X_test_s)  
y_pred = (y_prob > 0.5).astype(int)
```

9. Evaluation (LIKE PAGE 10)

```
print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)  
print("\nConfusion Matrix:\n", cm)
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
print("ROC AUC:", roc_auc_score(y_test, y_prob))
```

10. Confusion Matrix Heatmap (LIKE PAGE 11)

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("ANN Confusion Matrix")
plt.show()
```

```
# 11. ROC Curve (LIKE PAGE 12)
fpr, tpr, _ = roc_curve(y_test, y_prob)
```

```
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label="ANN ROC Curve")
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

OUTPUT

[Choose Files](#) Churn_Modelling.csv

Churn_Modelling.csv(text/csv) - 684858 bytes, last modified: 3/23/2026 - 100% done

Saving Churn_Modelling.csv to Churn_Modelling (8).csv

Dataset Shape: (10000, 14)

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

Model: "sequential_8"

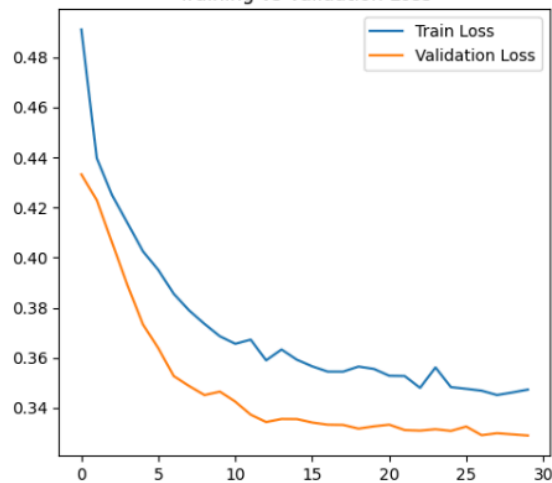
Layer (type)	Output Shape	Param #
dense_24 (Dense)	(None, 64)	768
dropout (Dropout)	(None, 64)	0
dense_25 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_26 (Dense)	(None, 16)	528
dense_27 (Dense)	(None, 1)	17

Total params: 3,393 (13.25 KB)
Trainable params: 3,393 (13.25 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/30
200/200 ————— 2s 3ms/step - accuracy: 0.7803 - loss: 0.4911 - val_accuracy: 0.8050 - val_loss: 0.4332
Epoch 2/30
200/200 ————— 1s 3ms/step - accuracy: 0.8050 - loss: 0.4397 - val_accuracy: 0.8244 - val_loss: 0.4229
Epoch 3/30
200/200 ————— 1s 3ms/step - accuracy: 0.8116 - loss: 0.4249 - val_accuracy: 0.8388 - val_loss: 0.4059
Epoch 4/30
200/200 ————— 1s 3ms/step - accuracy: 0.8208 - loss: 0.4138 - val_accuracy: 0.8487 - val_loss: 0.3889
Epoch 5/30
200/200 ————— 1s 3ms/step - accuracy: 0.8234 - loss: 0.4025 - val_accuracy: 0.8581 - val_loss: 0.3733
Epoch 6/30
200/200 ————— 1s 3ms/step - accuracy: 0.8294 - loss: 0.3950 - val_accuracy: 0.8631 - val_loss: 0.3638
Epoch 7/30
200/200 ————— 1s 3ms/step - accuracy: 0.8320 - loss: 0.3855 - val_accuracy: 0.8612 - val_loss: 0.3527
Epoch 8/30
200/200 ————— 1s 3ms/step - accuracy: 0.8403 - loss: 0.3789 - val_accuracy: 0.8612 - val_loss: 0.3487
Epoch 9/30
200/200 ————— 1s 3ms/step - accuracy: 0.8436 - loss: 0.3735 - val_accuracy: 0.8625 - val_loss: 0.3451
Epoch 10/30
200/200 ————— 1s 3ms/step - accuracy: 0.8422 - loss: 0.3686 - val_accuracy: 0.8612 - val_loss: 0.3465
Epoch 11/30
200/200 ————— 1s 3ms/step - accuracy: 0.8459 - loss: 0.3656 - val_accuracy: 0.8662 - val_loss: 0.3425
Epoch 12/30
200/200 ————— 1s 3ms/step - accuracy: 0.8430 - loss: 0.3672 - val_accuracy: 0.8662 - val_loss: 0.3373
Epoch 13/30
200/200 ————— 1s 3ms/step - accuracy: 0.8498 - loss: 0.3590 - val_accuracy: 0.8675 - val_loss: 0.3343

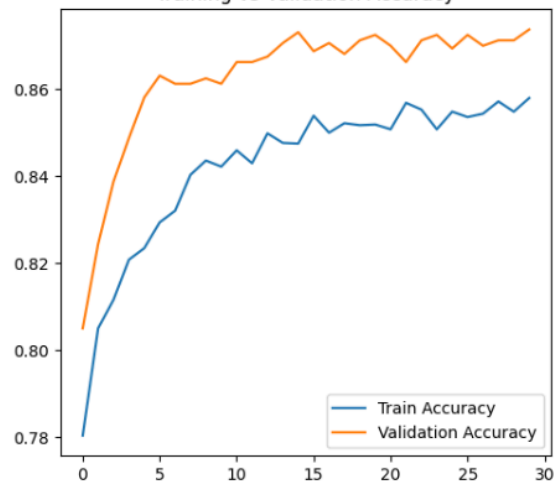
Epoch 25/30
200/200 ————— 1s 3ms/step - accuracy: 0.8548 - loss: 0.3483 - val_accuracy: 0.8694 - val_loss: 0.3308
Epoch 26/30
200/200 ————— 1s 3ms/step - accuracy: 0.8536 - loss: 0.3476 - val_accuracy: 0.8725 - val_loss: 0.3325
Epoch 27/30
200/200 ————— 1s 3ms/step - accuracy: 0.8544 - loss: 0.3468 - val_accuracy: 0.8700 - val_loss: 0.3291
Epoch 28/30
200/200 ————— 1s 3ms/step - accuracy: 0.8572 - loss: 0.3451 - val_accuracy: 0.8712 - val_loss: 0.3299
Epoch 29/30
200/200 ————— 1s 3ms/step - accuracy: 0.8548 - loss: 0.3462 - val_accuracy: 0.8712 - val_loss: 0.3294
Epoch 30/30
200/200 ————— 1s 3ms/step - accuracy: 0.8580 - loss: 0.3473 - val_accuracy: 0.8737 - val_loss: 0.3289

Training vs Validation Loss



63/63 ————— 0s 2ms/step

Training vs Validation Accuracy



Test Accuracy: 0.858

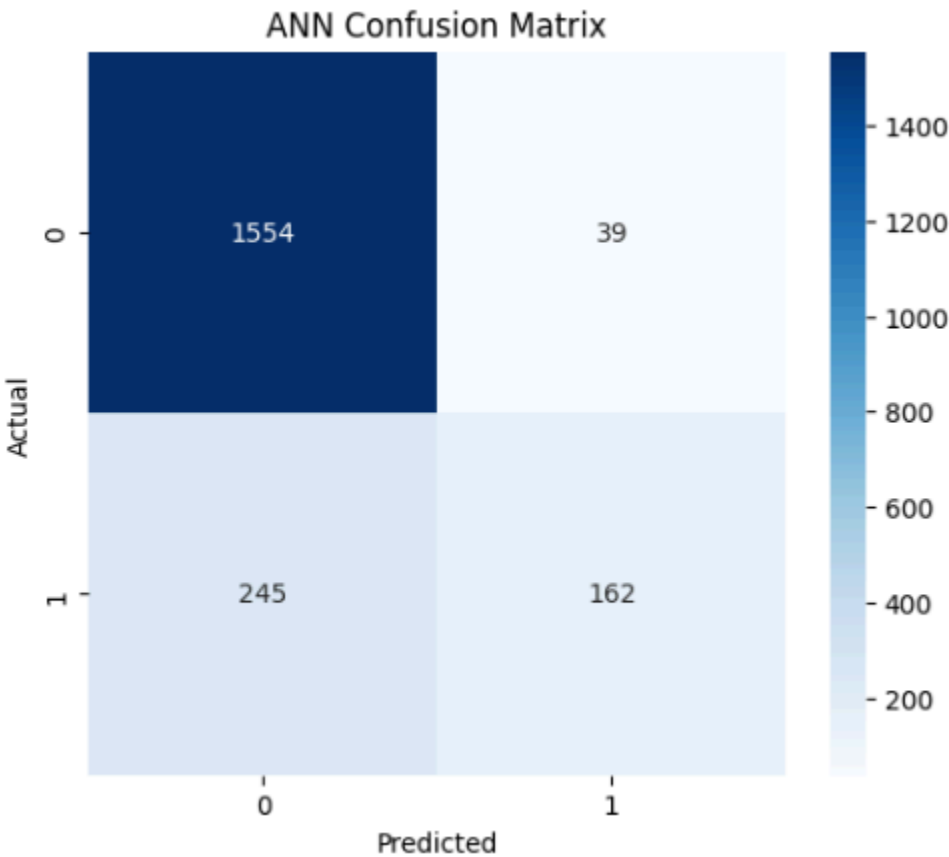
Confusion Matrix:

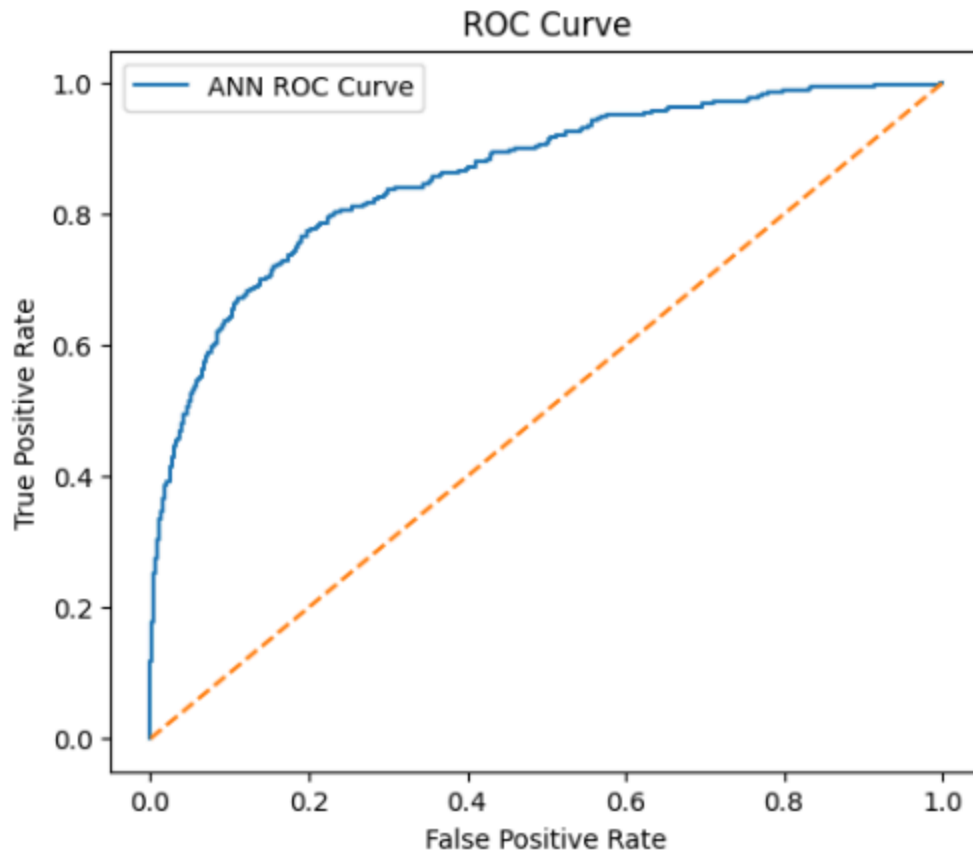
```
[[1554  39]
 [ 245 162]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.98	0.92	1593
1	0.81	0.40	0.53	407
accuracy			0.86	2000
macro avg	0.83	0.69	0.72	2000
weighted avg	0.85	0.86	0.84	2000

ROC AUC: 0.8609796236914882





CONCLUSION

- Artificial Neural Network is effective for classification problems like customer churn prediction
- The model captures complex relationships between features
- Feature scaling and proper architecture improve performance
- Dropout layers help reduce overfitting
- ANN provides a flexible and powerful approach for real-world prediction problems